CE195275 – Sequencing SAR ADC and Die Temperature Sensor with PSoC 4

Objective

This code example demonstrates the usage of the Sequencing SAR ADC Component with an injection channel and the Die Temperature Component.

Overview

This code example uses the Sequencing SAR ADC Component to measure input voltages and die temperature. The measurement results shown in the terminal program on a PC.

Requirements

Tool: PSoC® Creator™ 4.2

Programming Language: C (Arm® GCC 5.4.1 and Arm MDK 5.22)

Associated Parts: PSoC 4100, PSoC 4200, PSoC 4100 BLE, PSoC 4200 BLE, PSoC 4100M, PSoC 4200M, PSoC 4200L, PSoC 4100S, PSoC4100S Plus

Related Hardware: CY8CKIT-042, CY8CKIT-042-BLE, CY8CKIT-042-BLE-A, CY8CKIT-043, CY8CKIT-044, CY8CKIT-046, CY8CKIT-041-41XX, CY8CKIT-149

Hardware Setup

This code example supports kits, shown in Table 1. By default, this example project is configured to run on the CY8CKIT-042 development kit from Cypress Semiconductor. The project can be migrated to any supported kit by changing the target device with **Device Selector** called from the project's context menu. Refer to Table 1 for the target device name of your kit.

For all supported kits, the project includes control files to automatically assign pins with respect to the kit hardware connections during the project build. To change the pin assignments, override the control file selections in the Pin Editor of the Design Wide Resources by selecting the new port or pin number.

To run this example you will need two external voltage sources in range from 0 to the PSoC 4 Vdd voltage. You can use a voltage divider, potentiometer, etc.

For CY8CKIT-042 kit, connect the /UART_Tx/ pin (P0.5) of PSoC 4 to the UART Rx pin of KitProg (P12.6) with a jumper wire to use the KitProg USB-UART bridge. For other kits, the UART lines between PSoC 4 and KitProg are hard-wired on kit's board.

Table 1. Supported Kits, Devices, Pin Assignments

Kit	Series	Device	Pin Assignments		
			Pin_Vin0	Pin_Vin1	/UART_tx/
CY8CKIT-042	PSoC 4200	CY8C4245AXI-483	P2[0]	P2[1]	P0[5]
CY8CKIT-042-BLE	PSoC 4200 BLE	CY8C4247LQI-BL483	P3[0]	P3[1]	P1[5]
CY8CKIT-042-BLE-A	PSoC 4200 BLE	CY8C4248LQI-BL583			
CY8CKIT-043	PSoC 4200M	CY8C4247AZI-M485	P2[0]	P2[1]	P7[1]
CY8CKIT-044	PSoC 4200M	CY8C4247AZI-M485	P2[0]	P2[1]	P7[1]
CY8CKIT-046	PSoC 4200L	CY8C4248BZI-L489	P2[0]	P2[1]	P3[1]
CY8CKIT-041-41XX	PSoC 4100S	CY8C4146AZI-S433	P2[3]	P2[4]	P0[5]
CY8CKIT-149	PSoC 4100S Plus	CY8C4147AZI-S475	P2[3]	P2[4]	P7[1]

Software Setup

For this code example, any terminal software can be used: HyperTerminal, Bray's Terminal, PuTTY, etc. The configuration of the terminal described in the "Operation" section.

Operation

1. Plug the kit board into your computer's USB port.
2. Build the project and program it into the PSoC 4 device. Choose **Debug** > **Program**. For more information on the device programming, see the PSoC Creator Help.

Note: By default, the ADC_SAR_Seq Component customizer has the Vref select parameter set to **Internal 1.024 volts**. PSoC 4100S and PSoC 4100S Plus devices have an internal reference of 1.2V; therefore, the Vref select parameter must be set to **Internal Vref**. Building an example project without changing the default Vref select parameter value for these devices will cause a building error – "Error in component: ADC_SAR_Seq. The selected type of voltage reference is not supported for the current device type".

3. Run any COM terminal program. Set the terminal parameters: **Baud Rate** – 115200, **Data Bits** – 8, **Parity** – none, **Stop Bits** – 1 and connect to the corresponding port.

Note: To find COM port number, open the Device Manager in your PC, find the device **KitProg USBUART** or **KitProg2 USBUART** under **Ports** (COM & LPT), and note the port number.

4. Connect the voltage sources to the ADC voltage inputs: Pin_Vin0 and Pin_Vin1. For the CY8CKIT-041 kit, use an on-board potentiometer to change the voltage value on the Pin_Vin1 input.

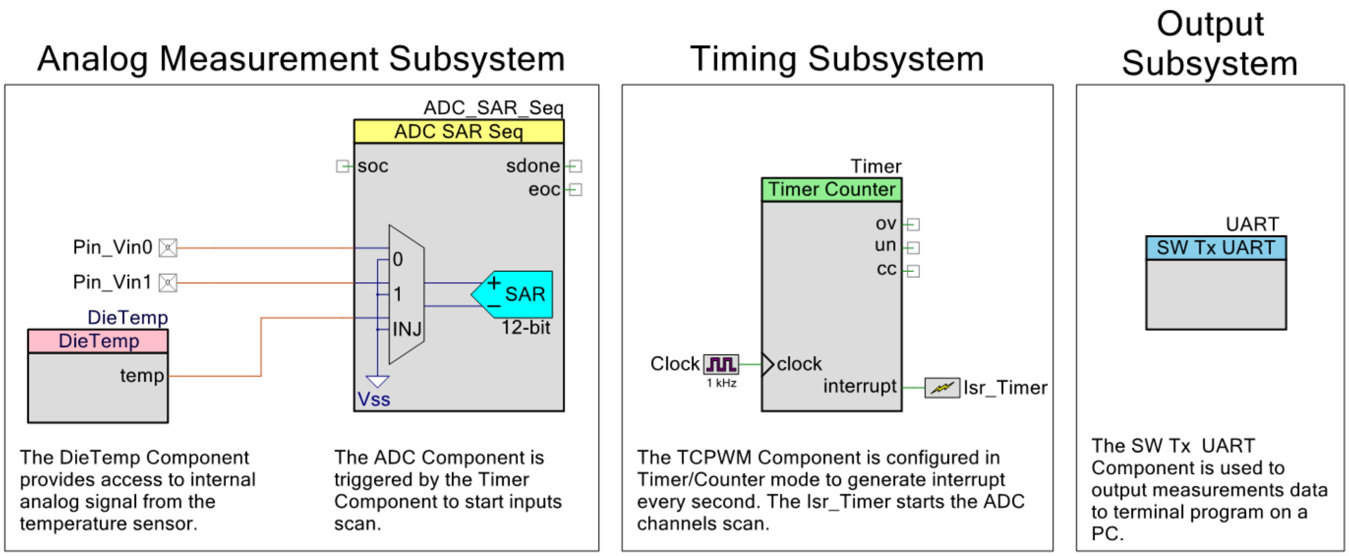
Note: The ADC measurement result will be out of range (1024 mV) for a voltage larger than 1.024 V. You may use an external voltage divider to get the expected voltage in the range (0 - 1.024 V). For the PSoC 4100S and PSoC 4100S Plus devices (CY8CKIT-041 and CY8CKIT-149 kits), the upper voltage limit will be 1.2 V.

5. Observe the inputs voltage and device temperature values on the terminal program.

Design and Implementation

The PSoC hardware design is shown in Figure 1.

Figure 1. PSoC Creator Project Schematic



The ADC_SAR_Seq uses: two sequenced channels to measure voltage values from the input pins; an injection channel to measure voltage values from the DieTemp Component. To increase the ADC measurement accuracy Averaging Mode is used for all input channels.

The DieTemp Component provides access only to internal analog signal from the temperature sensor and must be used in conjunction with an ADC to produce a digital value. Temperature changes slowly; therefore, this signal is used with the injection channel and samples values infrequently compared to the sequenced channel.

The TCPWM Component is configured in Timer/Counter mode to generate interrupt every second. Timer's interrupt handler starts the sequenced channels scan every second and enables the injection channel every fifth sequenced scan. You can change sequenced and injection channel scan periods by changing the Timer Component period in the Component customizer and `ADC_INJ_PERIOD` value in the *main.c* file.

The UART Component is used to output voltage and temperature measurements.

The firmware performs these functions:

1. Initializes the UART, ADC, and Timer.
2. Starts the ADC conversion.
3. Waits for the converted results.
4. Calculates voltage and temperature values.
5. Prints the calculated values into the UART.

Components and Settings

Table 2 lists the PSoC Creator Components used in this example, how they are used in the design, and the non-default settings required so they function as intended.

Table 2. PSoC Creator Components

Component	Instance Name	Purpose	Non-default Settings
Sequencing SAR ADC	ADC_SAR_Seq	Measures voltage values from the input pins and DieTemp Component.	See Figure 2 and Figure 3
Die Temperature	DieTemp	Provides access to internal analog signal from the temperature sensor.	Default settings only
Analog Pin	Pin_Vin0 Pin_Vin1	The input pins of the ADC.	Default settings only
Timer Counter (TCPWM mode)	Timer	Starts the ADC scan with desired period.	Period: 999
Clock	Clock	The clock source for the Timer Component.	Frequency: 1 kHz
Software Transmit UART	UART	Sends measurement results to the terminal program on the PC.	Default settings only

For information on the hardware resources used by a Component, see the Component datasheet.

Figure 2 and Figure 3 show the ADC_SAR_Seq Component configuration with the highlighted non-default settings.

Figure 2. ADC_SAR_Seq General Settings Tab

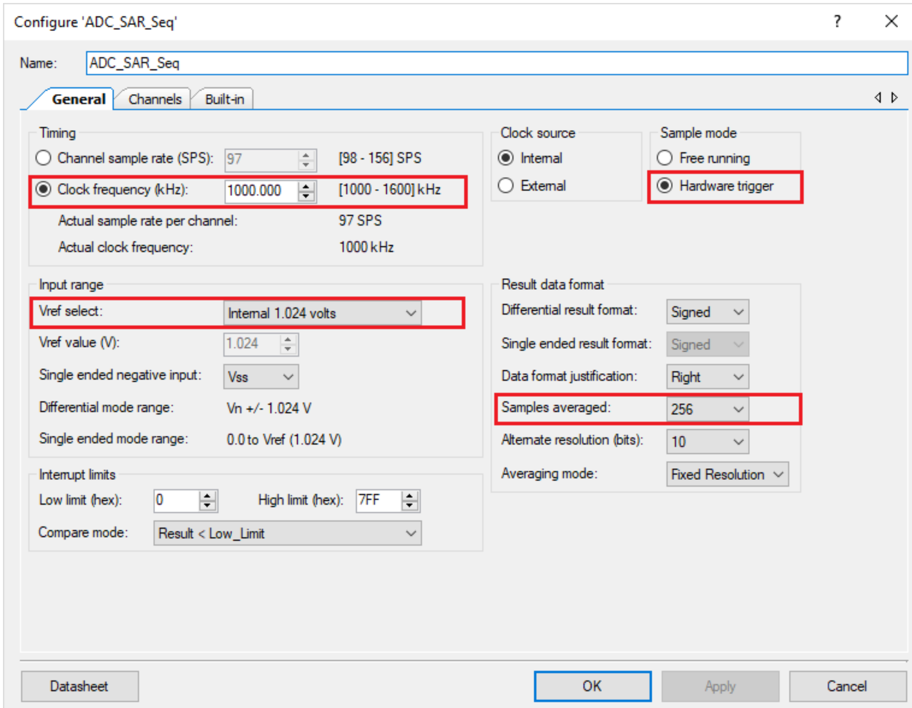
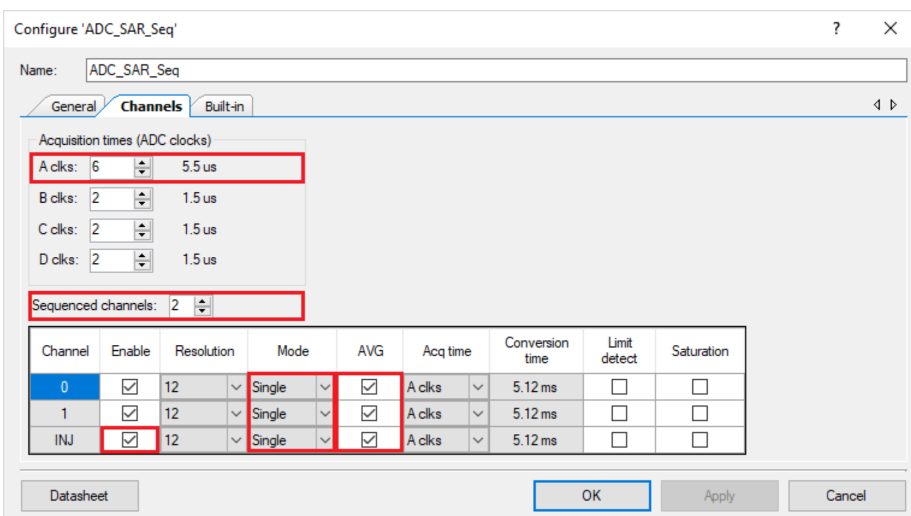


Figure 3. ADC_SAR_Seq Channels Settings Tab



Reusing This Example

This example is designed for the kits listed in Table 1. To port the design to a different PSoC 4 device and/or kit, change the target device using **Device Selector** and update the Tx pin assignment in the **Design Wide Resources Pins** settings as needed.

Related Documents

Application Notes	
AN79953 – Getting Started with PSoC 4	Introduces the PSoC 4 architecture and development tools.
PSoC Creator Component Datasheets	
Sequencing SAR ADC	Supports Sequencing SAR ADC to perform analog to digital conversions
Die Temperature	Supports on-chip temperature sensor
Timer Counter (TCPWM mode)	Supports TCPWM blocks
Pins	Supports connection of hardware resources to physical pins
Device Documentation	
PSoC 4 Datasheets	PSoC 4 Technical Reference Manuals
Development Kit (DVK) Documentation	
PSoC 4 Kits	

Document History

Document Title: CE195275 – Sequencing SAR ADC and Die Temperature Sensor with PSoC 4

Document Number: 001-95275

Revision	ECN	Orig. of Change	Submission Date	Description of Change
**	5946241	MYKZTMP1	04/06/2018	New code example

Worldwide Sales and Design Support

Cypress maintains a worldwide network of offices, solution centers, manufacturer's representatives, and distributors. To find the office closest to you, visit us at Cypress Locations.

Products

Arm® Cortex® Microcontrollers	cypress.com/arm
Automotive	cypress.com/automotive
Clocks & Buffers	cypress.com/clocks
Interface	cypress.com/interface
Internet of Things	cypress.com/iot
Memory	cypress.com/memory
Microcontrollers	cypress.com/mcu
PSoC	cypress.com/psoc
Power Management ICs	cypress.com/pmic
Touch Sensing	cypress.com/touch
USB Controllers	cypress.com/usb
Wireless Connectivity	cypress.com/wireless

PSoC® Solutions

PSoC 1 | PSoC 3 | PSoC 4 | PSoC 5LP | PSoC 6 MCU

Cypress Developer Community

Community Forums | Projects | Videos | Blogs | Training | Components

Technical Support

cypress.com/support

All other trademarks or registered trademarks referenced herein are the property of their respective owners.

Cypress Semiconductor
198 Champion Court
San Jose, CA 95134-1709

© Cypress Semiconductor Corporation, 2018. This document is the property of Cypress Semiconductor Corporation and its subsidiaries, including Spansion LLC ("Cypress"). This document, including any software or firmware included or referenced in this document ("Software"), is owned by Cypress under the intellectual property laws and treaties of the United States and other countries worldwide. Cypress reserves all rights under such laws and treaties and does not, except as specifically stated in this paragraph, grant any license under its patents, copyrights, trademarks, or other intellectual property rights. If the Software is not accompanied by a license agreement and you do not otherwise have a written agreement with Cypress governing the use of the Software, then Cypress hereby grants you a personal, non-exclusive, nontransferable license (without the right to sublicense) (1) under its copyright rights in the Software (a) for Software provided in source code form, to modify and reproduce the Software solely for use with Cypress hardware products, only internally within your organization, and (b) to distribute the Software in binary code form externally to end users (either directly or indirectly through resellers and distributors), solely for use on Cypress hardware product units, and (2) under those claims of Cypress's patents that are infringed by the Software (as provided by Cypress, unmodified) to make, use, distribute, and import the Software solely for use with Cypress hardware products. Any other use, reproduction, modification, translation, or compilation of the Software is prohibited.

TO THE EXTENT PERMITTED BY APPLICABLE LAW, CYPRESS MAKES NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS DOCUMENT OR ANY SOFTWARE OR ACCOMPANYING HARDWARE, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. No computing device can be absolutely secure. Therefore, despite security measures implemented in Cypress hardware or software products, Cypress does not assume any liability arising out of any security breach, such as unauthorized access to or use of a Cypress product. In addition, the products described in these materials may contain design defects or errors known as errata which may cause the product to deviate from published specifications. To the extent permitted by applicable law, Cypress reserves the right to make changes to this document without further notice. Cypress does not assume any liability arising out of the application or use of any product or circuit described in this document. Any information provided in this document, including any sample design information or programming code, is provided only for reference purposes. It is the responsibility of the user of this document to properly design, program, and test the functionality and safety of any application made of this information and any resulting product. Cypress products are not designed, intended, or authorized for use as critical components in systems designed or intended for the operation of weapons, weapons systems, nuclear installations, life-support devices or systems, other medical devices or systems (including resuscitation equipment and surgical implants), pollution control or hazardous substances management, or other uses where the failure of the device or system could cause personal injury, death, or property damage ("Unintended Uses"). A critical component is any component of a device or system whose failure to perform can be reasonably expected to cause the failure of the device or system, or to affect its safety or effectiveness. Cypress is not liable, in whole or in part, and you shall and hereby do release Cypress from any claim, damage, or other liability arising from or related to all Unintended Uses of Cypress products. You shall indemnify and hold Cypress harmless from and against all claims, costs, damages, and other liabilities, including claims for personal injury or death, arising from or related to any Unintended Uses of Cypress products.

Cypress, the Cypress logo, Spansion, the Spansion logo, and combinations thereof, WICED, PSoC, CapSense, EZ-USB, F-RAM, and Traveo are trademarks or registered trademarks of Cypress in the United States and other countries. For a more complete list of Cypress trademarks, visit cypress.com. Other names and brands may be claimed as property of their respective owners.